



Comparison of a Classical RAG Pipeline and an Agent-Based Approach for Intelligent Knowledge Retrieval Across Enterprise Documentation Sources

Bachelorarbeit

submitted in partial fulfilment of the requirements for the degree of

Bachelor of Science

in the degree program New Study Informatik

at the Baden-Württemberg Cooperative State University Stuttgart

by

Liam Thomson

2026

Matrikulation Number, Course	3248571, DHBW-TINFO23
Training Company	Accenture, Kronberg im Taunus
Supervisors	Piotr Preuss
	Prof. Dr. Katja Wengler
Examiner	Dr. Prof. Kay Berkling

Confidentiality Notice

The present Bachelorarbeit titled *Comparison of a Classical RAG Pipeline and an Agent-Based Approach for Intelligent Knowledge Retrieval Across Enterprise Documentation Sources* is classified as confidential and is submitted solely for examination purposes in the New Study Informatik program at the Baden-Württemberg Cooperative State University Stuttgart. Any inspection or publication — including partial excerpts of the thesis — requires prior approval from Accenture.

Declaration

I hereby declare under oath:

1. that I have completed the present Bachelorarbeit on the topic *Comparison of a Classical RAG Pipeline and an Agent-Based Approach for Intelligent Knowledge Retrieval Across Enterprise Documentation Sources* independently and without external assistance;
2. that I have indicated the adoption of literal quotations from the literature and the use of the ideas of other authors at the corresponding points within the work;
3. that I have not submitted this Bachelorarbeit for any other examination.

I am aware that a false declaration will have legal consequences.

Baden-Württemberg Cooperative State University Stuttgart, 2026

Liam Thomson

Abstract

Table of Contents

1 Introduction **1**
- 1.1 The Enterprise Knowledge Problem . . . 1
- 1.2 The Predecessor Project and Its Limitations . . . 2
- 1.3 Two Paradigms Under Comparison . . . 3
- 1.4 Research Questions . . . 4
- 1.5 Scope and Delimitations . . . 5

2 Theoretical Foundations **6**
- 2.1 Enterprise Knowledge Retrieval . . . 6
- 2.2 Retrieval-Augmented Generation . . . 8
- 2.3 Agent-Based Information Retrieval . . . 10
- 2.4 Evaluation Methodology . . . 13

3 System Design **18**
- 3.1 Shared Data Layer . . . 19
  - 3.1.1 Data Source Selection and Justification . . . 19
  - 3.1.2 Unified Document Schema . . . 19
  - 3.1.3 Ingestion Data Flow . . . 19
- 3.2 Classical RAG Pipeline Architecture . . . 19
  - 3.2.1 Pipeline Stages . . . 19
  - 3.2.2 Chunking Strategy . . . 19
  - 3.2.3 Embedding Model and Vector Store . . . 19
  - 3.2.4 Retrieval and Reranking . . . 19
- 3.3 Agent-Based System Architecture . . . 19
  - 3.3.1 The Agent Loop . . . 19
  - 3.3.2 Tools and Source Routing . . . 19
  - 3.3.3 Orchestration Logic . . . 19
- 3.4 Comparability Design Decisions . . . 19

4 Data and Evaluation Setup **20**
- 4.1 Dataset Construction . . . 21
  - 4.1.1 Data Collection per Source Type . . . 21
  - 4.1.2 Preprocessing and Normalisation . . . 21

4.1.3 Corpus Statistics . . . 21
4.2 Evaluation Query Set . . . 21
4.2.1 Synthetic QA Generation . . . 21
4.2.2 Query Taxonomy . . . 21
4.2.3 Manual Review and Filtering . . . 21
4.2.4 Final Query Set Statistics . . . 21
4.3 Evaluation Metrics Definition . . . 21
4.3.1 RAGAS Scores . . . 21
4.3.2 BERTScore and ROUGE-L . . . 21
4.3.3 Latency and Token Cost . . . 21
4.3.4 Agent-Specific Metrics . . . 21
4.3.5 Complexity and Maintainability Rubric . . . 21

5 Implementation **22**
5.1 Development Environment and Tooling . . . 22
5.2 RAG Pipeline Implementation . . . 22
5.2.1 Chunking and Embedding . . . 22
5.2.2 Retrieval and Reranking . . . 22
5.2.3 Failure Modes and Resolutions . . . 22
5.3 Agent Implementation . . . 22
5.3.1 Agent Loop . . . 22
5.3.2 Tool Implementations . . . 22
5.3.3 System Prompt and Routing Behaviour . . . 22
5.4 Logging and Instrumentation . . . 22

6 Evaluation Results **23**
6.1 Retrieval and Generation Quality . . . 23
6.2 Latency and Cost . . . 23
6.3 Implementation Complexity and Maintainability . . . 23
6.4 Robustness Observations . . . 23

7 Discussion **24**
7.1 Answering the Research Questions . . . 24
7.2 Trade-Off Analysis . . . 24
7.3 Limitations . . . 24
7.4 Connection to Prior Work . . . 24

8 Conclusion **25**
8.1 Summary of Findings . . . 25
8.2 Practical Recommendation . . . 25
8.3 Future Work . . . 25

Figures i

Tables ii

Listings iii

References iv

1 Introduction

1.1 The Enterprise Knowledge Problem

Knowledge in large organisations is rarely located in one place. Relevant information is typically distributed across issue trackers, collaborative wikis, document repositories, and local file systems — each with its own search interface, data model, and access pattern. When a team member needs a precise, context-aware answer to an operational or client-related question, locating that answer requires either navigating multiple systems in sequence or relying on the tacit knowledge of a colleague who has already done so. This dependency on individual expertise creates fragility: when key contributors are unavailable, institutional knowledge becomes effectively inaccessible.

The problem is compounded by the structural diversity of enterprise sources. Issue tracker entries — such as Jira tickets — are short, structured, and dominated by metadata fields: status, assignee, priority, and linked items. Collaborative wiki pages — such as Confluence articles — are longer, discursive, and typically organised by project or function. Whitepapers and policy documents stored in SharePoint or similar repositories are formally structured but may span dozens of pages, with the most relevant passage buried in a single subsection. A retrieval system that performs well on one of these source types does not necessarily perform well on the others, because the retrieval semantics, document length distribution, and appropriate context window differ substantially between them.

The aggregate cost of this fragmentation is well-recognised in practice. Time spent locating information rather than acting on it reduces operational efficiency; answers derived from incomplete retrieval produce decisions based on partial evidence; and the accumulation of tacit knowledge in individuals rather than in shared, queryable systems represents a structural risk for organisations with high staff turnover or geographically

distributed teams. Automated knowledge retrieval systems, if they can operate reliably across heterogeneous sources, offer a direct remedy to each of these failure modes.

1.2 The Predecessor Project and Its Limitations

A previous project at Accenture addressed this challenge by extending the Ask Your Template (AYT) platform — an internal Retrieval-Augmented Generation (RAG) pipeline — with integration for Jira and Confluence. In that system, knowledge objects from both platforms were ingested, normalised, split into fixed-size text segments, converted into dense vector embeddings, and stored in a searchable index. At query time, a user's question was embedded and matched against the stored vectors to retrieve the most similar segments; those segments were then passed to a large language model (LLM) as context, and a natural-language answer was generated in response.

The AYT integration demonstrated that semantic retrieval over enterprise knowledge sources is technically feasible and useful in an operational context. The evaluation of that system also surfaced a set of structural limitations that motivate the present thesis directly. Chunking strategy — the method by which documents are divided into retrievable segments — proved difficult to calibrate: segments that were too short lost the surrounding context necessary for coherent answers, while segments that were too long introduced irrelevant content into the retrieval signal. Source-specific retrieval quality varied significantly, with Jira tickets and Confluence articles responding differently to the same embedding and retrieval configuration despite nominally belonging to the same index. The setup and maintenance burden was non-trivial, and extending the system to additional data sources required manual reconfiguration of the ingestion pipeline for each new source type.

These limitations are not unique to the AYT implementation; they are recognised failure modes of classical RAG pipelines applied to heterogeneous source collections (Gao, Xiong, Gao, Jia, Pan, Bi, Dai, Sun and Wang, 2023). A fixed retrieval pipeline — one that applies the same chunking, embedding, and retrieval strategy uniformly across all source types — may be structurally ill-suited to environments where source documents differ substantially in length, format, and information density. This structural mismatch is the central motivation for examining an agent-based alternative.

1.3 Two Paradigms Under Comparison

Retrieval-Augmented Generation addresses the knowledge limitation of LLMs by supplementing generation with externally retrieved content. In a classical RAG pipeline, a query is encoded as a vector, matched against a pre-built index of document embeddings, and the most relevant chunks are provided to the LLM as context for generating an answer. The approach grounds language model outputs in external knowledge, reducing factual hallucination and enabling knowledge updates without model retraining (Lewis, Perez, Piktus, Petroni, Karpukhin, Goyal, Küttler, Lewis, Yih, Rocktäschel, Riedel and Kiela, 2020). Classical RAG follows a fixed sequential structure: the retrieval step always occurs, always queries the same index, and always returns a fixed number of chunks, regardless of the query's nature, complexity, or appropriate source.

Agent-based systems represent a different paradigm. Rather than following a predetermined retrieval pipeline, an LLM-based agent reasons explicitly about what information is needed, selects the appropriate tool or data source to query, executes that query, observes the result, and iterates until the query is satisfactorily resolved (Yao, Zhao, Yu, Du, Shafran, Narasimhan and Cao, 2023). In a multi-source enterprise context, this means the agent can route a question about a software defect to an issue tracker retriever, a question about a deployment procedure to a documentation retriever, and a question about organisational standards to a PDF retriever — all within the same reasoning loop. Source routing, task decomposition, and iterative retrieval become explicit, observable steps in the agent's decision process rather than implicit assumptions baked into a fixed pipeline.

The two paradigms differ substantially in their design philosophy and expected performance characteristics. Classical RAG optimises for simplicity and predictability: a single retrieval step, a fixed context window, and a deterministic pipeline that is straightforward to deploy and monitor. Agent-based retrieval optimises for adaptability: dynamic source selection, multi-step reasoning, and the ability to refine a retrieval strategy based on intermediate observations. Whether this adaptability produces measurably better answers — and at what cost in latency, token usage, and implementation effort — is the central empirical question this thesis addresses.

1.4 Research Questions

Four research questions structure the comparative evaluation:

1. *How does a classical RAG pipeline compare to an agent-based approach in terms of latency, cost, and retrieval performance in enterprise environments?*

   This question establishes the baseline empirical comparison. Latency and cost are the primary operational constraints in enterprise deployment; retrieval performance — measured through context relevance, answer faithfulness, and answer relevance — determines whether either system is fit for the task at all. Without this foundation, comparisons of architectural sophistication are not grounded in observable outcomes.

2. *To what extent does an agent-based system improve source selection and answer quality compared to RAG?*

   Source selection accuracy — whether the system queries the appropriate document type for a given question — is a capability that agent-based systems are specifically designed to support but that classical RAG pipelines, by design, cannot implement. This question tests whether that architectural advantage translates into measurable quality gains, particularly on queries that span multiple source types or require iterative retrieval.

3. *What are the differences in implementation complexity and maintainability between both approaches?*

   A system that produces marginally better answers but requires substantially more effort to build, configure, and extend may not represent the right architectural choice for the majority of enterprise teams. By making implementation complexity an explicit evaluation dimension — assessed through code volume, configuration depth, and the effort required to onboard a new data source — this question ensures that the comparison is practically actionable.

4. *Under which conditions is each approach preferable in practice?*

   The first three questions yield comparative measurements across individual dimensions; the fourth synthesises them into a conditional recommendation. Given a specific combination of corpus characteristics, query complexity distribution,

team expertise, and resource constraints, which architecture is the more appropriate choice? This question is the applied output of the thesis and its primary contribution to practitioners facing this architectural decision.

1.5 Scope and Delimitations

This thesis focuses on the comparative evaluation of two retrieval architectures — a classical RAG pipeline and a ReAct-based agent — applied to a shared corpus of enterprise-proxy documents. Both systems are implemented as functional research prototypes and evaluated under identical conditions; the findings are intended to inform architectural decisions, not to constitute a production deployment recommendation.

Several topics fall outside the scope of this work. Production hardening — including security, access control, rate limiting, and audit logging — is not addressed; both systems are evaluated in a controlled, single-user experimental environment. User studies and human evaluation of answer quality are excluded; evaluation relies on automated metrics supplemented by LLM-as-judge scoring, a methodology validated by Es, James, Anke and Schockaert (2023) as sufficiently correlated with human judgements for comparative evaluation purposes. Multi-agent coordination — systems in which multiple specialist agents collaborate on a single query — is beyond the scope of the implementation; the agent system uses a single reasoning loop with multiple tools. Fine-tuning of either the LLM or the embedding model on enterprise-specific data is not performed; both systems rely on general-purpose pre-trained models.

The document corpus consists of three open, publicly available source types: GitHub Issues, Markdown documentation, and PDF whitepapers. These are selected as structural proxies for the enterprise source types — Jira tickets, Confluence wiki pages, and SharePoint documents — that motivated the original AYT project. This proxy design enables reproducible academic evaluation without requiring proprietary data access. It introduces a generalisability limitation, which is addressed in Chapter 7.

2 Theoretical Foundations

2.1 Enterprise Knowledge Retrieval

Enterprise knowledge retrieval encompasses the challenge of surfacing relevant, context-aware information from collections of heterogeneous documents distributed across organisational systems. Modern enterprises accumulate knowledge across structurally distinct source types—issue trackers, collaborative wikis, and document repositories—each of which imposes different constraints on how information is stored, queried, and consumed. The resulting heterogeneity is not merely a matter of file format; it reflects fundamental differences in document length, information density, structural formality, and the semantics of the queries users direct at each source.

The three source types employed in this thesis illustrate the range of structural variation encountered in practice. Issue tracker entries, represented here by GitHub Issues as a proxy for Jira tickets, are short and metadata-rich, with discrete fields for status, labels, and assignees alongside free-text comment threads. Collaborative wiki pages, represented by Markdown documentation as a proxy for Confluence, are discursive and medium-length, organised by human-authored headings into flowing prose. Formal document repositories, represented by PDF whitepapers as a proxy for SharePoint, contain long, hierarchically structured texts with dense technical content and standardised sections. A retrieval approach that treats these source types uniformly risks systematic degradation on at least one category, as the predecessor project AYT demonstrated when classical retrieval-augmented generation (RAG) produced uneven results across the same three structural classes.

Latency expectations compound the difficulty of the retrieval problem. Enterprise users tolerate response times that vary with query complexity: simple factual lookups — retrieving the status of a known issue or locating a specific configuration parameter — carry an implicit expectation of near-instantaneous response, whereas cross-document

synthesis queries, which require combining evidence from multiple sources, are accepted with longer wait times. A retrieval system must therefore balance thoroughness against speed in a manner that is sensitive to query type, not only to document volume.

Traditional enterprise search has long relied on lexical matching methods, of which the BM25 ranking function is the canonical representative. BM25 scores documents by term frequency and inverse document frequency, yielding high precision for queries that share surface tokens with the target document. Its limitations emerge when the query and the relevant passage share meaning without sharing vocabulary: a query for "deployment instructions" will fail to retrieve a document whose relevant passage reads "steps to install and configure the service," because no term overlap exists. This vocabulary mismatch problem is a persistent source of recall loss in operational retrieval systems.

Federated search addresses a different dimension of the enterprise retrieval problem by coordinating queries across multiple disparate systems and aggregating the results into a unified response. It solves the organisational challenge of querying siloed repositories without requiring data migration, but it does not address the semantic gap: each constituent system still applies lexical matching internally, and the aggregation layer has no mechanism for resolving conceptual relationships across sources. Federated search thus increases coverage without improving the quality of individual matches, leaving cross-source synthesis queries — those that require reconciling information distributed across an issue thread, a wiki page, and a whitepaper — beyond its reach.

The limitations of both approaches converge on a shared root cause: neither represents documents or queries as continuous semantic objects. Keyword search operates on discrete tokens; federated search multiplies the number of token-matching engines without changing the underlying representation. Semantic retrieval methods, which map documents and queries into dense vector representations where proximity reflects meaning rather than surface form, address this root cause directly. The two paradigms examined in this thesis — classical RAG and agent-based retrieval — both build on semantic representations as their foundation, and the comparative evaluation consequently begins from a baseline that lexical search cannot achieve.

2.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation extends the capabilities of large language models (LLMs) by grounding generation in dynamically retrieved external knowledge, rather than relying solely on parameters encoded during training (Lewis et al., 2020). The original formulation by Lewis et al. (2020) decomposes the system into two learnable components: a retriever, parameterised as a Dense Passage Retriever (DPR) that estimates the probability of a document given a query, and a generator, parameterised as a sequence-to-sequence model that conditions its output on both the query and the retrieved passages. A critical architectural property of this design is that the non-parametric retrieval index can be updated or expanded without retraining the generator, making the system's knowledge base directly inspectable and modifiable (Lewis et al., 2020).

The core RAG pipeline proceeds across three sequential phases: indexing, retrieval, and generation. During indexing, source documents are segmented into chunks, encoded by an embedding model into dense vector representations, and stored in a vector index alongside their source metadata. At query time, the retrieval phase encodes the user query using the same embedding model, computes similarity against the stored vectors — typically using cosine similarity or inner product — and returns the top-k most relevant chunks. These retrieved passages are then concatenated with the original query and passed to the generator as a prompt context, from which the final response is produced. The generator therefore operates on a composed input that merges parametric knowledge with non-parametric evidence, reducing the risk of hallucination on knowledge-intensive tasks.

The RAG landscape has since diversified substantially, and Gao et al. (2023) organise this development into three paradigms: Naive RAG, Advanced RAG, and Modular RAG. Naive RAG implements the core pipeline described above without modification — a single retrieval step, no query transformation, and no post-retrieval filtering — and serves as the tractable baseline against which improvements are measured. However, Naive RAG exhibits well-documented failure modes: retrieval precision is often low, with irrelevant chunks included in the context, while recall gaps mean that relevant passages may not surface at all (Gao et al., 2023). These deficiencies propagate directly into generation, where the presence of contradictory or off-topic retrieved content can cause the model to produce responses that conflict with the retrieved evidence.

Advanced RAG addresses these failure modes by introducing pre-retrieval and post-retrieval processing stages around the core pipeline. Pre-retrieval techniques — including query rewriting, query expansion, and Hypothetical Document Embedding (HyDE), where the model generates a synthetic answer used as the retrieval query — improve the semantic alignment between the query and the indexed content. Post-retrieval techniques, most notably reranking with a cross-encoder model, reorder the initial top-k candidates by scoring each passage against the query jointly rather than independently, yielding substantially better relevance ordering (Gao et al., 2023). Context compression further reduces noise by truncating or filtering retrieved chunks to the most answer-relevant spans before they are passed to the generator. The classical pipeline implemented in this thesis is positioned as an Advanced RAG system: it performs query-level embedding, vector retrieval, cross-encoder reranking, and context compression, but applies a single fixed retrieval step without source routing or adaptive retrieval decisions.

Modular RAG deconstructs the pipeline further into interchangeable components and introduces new capabilities not present in either earlier paradigm. Routing directs different query types to different retrieval strategies or data sources; scheduling determines whether retrieval is needed at all for a given query; and adaptive retrieval enables the system to retrieve multiple times, conditioning each subsequent query on prior observations (Gao et al., 2023). These capabilities bring Modular RAG into conceptual proximity with agent-based systems, in which source routing and adaptive retrieval are externalised as explicit reasoning steps rather than embedded as pipeline parameters.

The effectiveness of a RAG system depends considerably on chunking strategy, which determines the granularity and coherence of the indexed units. Fixed-size chunking is computationally cheap but routinely splits passages at semantically arbitrary boundaries, degrading retrieval precision. Sentence-level chunking respects natural language units yet produces short chunks that may lack sufficient context for accurate relevance scoring. The Small2Big — or hierarchical — strategy offers the strongest balance: small chunks are indexed and retrieved for retrieval precision, but the generator receives the surrounding parent chunk as context, preserving local coherence (Gao et al., 2023). Sliding window chunking with configurable overlap reduces the risk of splitting relevant spans across adjacent chunks and remains a practical complement to sentence-level segmentation.

Embedding model choice determines the representational capacity of the retrieval index. Sparse retrieval methods, such as BM25, operate on exact lexical overlap and serve as strong baselines for short, precise queries, but generalise poorly to paraphrase or synonym variation. Dense retrieval models — including the BGE and E5 families, evaluated comparatively on the Massive Text Embedding Benchmark (MTEB) — learn semantic representations that capture meaning beyond surface form, enabling retrieval across queries that share intent but not vocabulary (Lewis et al., 2020). In practice, hybrid retrieval combining sparse and dense signals can outperform either method alone on heterogeneous corpora.

Despite these enhancements, Advanced RAG remains a static pipeline: retrieval is always performed, always from a single index, and always at a fixed point in the generation process. This design assumption becomes a liability when the information need is ambiguous, when multiple retrieval steps are required to integrate evidence across documents, or when retrieval is unnecessary and introduces noise. Asai, Wu, Wang, Sil and Hajishirzi (2023) demonstrate that indiscriminate retrieval, regardless of whether the query requires it, reduces model performance; removing adaptive retrieval decisions from their system causes accuracy to fall from 54.9 to 42.6 on PopQA. Jiang, Xu, Gao, Sun, Liu, Dwivedi-Yu, Yang, Callan and Neubig (2023) show that a single retrieval step is insufficient for long-form generation tasks where information needs evolve across the output, and that active, forward-looking retrieval outperforms a fixed single-step baseline by a substantial margin on multi-hop reasoning tasks. Taken together, these findings suggest that the architectural ceiling of static retrieval pipelines motivates a fundamentally different approach — one in which retrieval decisions are made dynamically by a system capable of selecting sources, composing multi-step queries, and adjusting its strategy based on intermediate results.

2.3 Agent-Based Information Retrieval

Agent-based systems extend language model capabilities by embedding the model within a decision-making cycle that interleaves reasoning, action, and environmental feedback. Where a classical RAG pipeline executes a fixed sequence — retrieve once, then generate — an agent-based system treats retrieval as one of many possible actions, selected dynamically based on the current query state and prior observations. Xi, Chen, Guo, He, Ding, Hong, Zhang, Wang, Jin, Zhou, Zheng, Fan, Wang, Xiong, Liu, Zhou,

Wang, Jiang, Zou, Liu, Yin, Dou, Weng, Cheng, Zhang, Qin, Zheng, Qiu, Huang and Gui (2023) define this class of system through four distinguishing properties: autonomy, in that the agent operates without direct human intervention at each step; reactivity, in that it responds to observations returned by tools; pro-activeness, in that it pursues a goal across multiple steps; and social ability, in that it can in principle coordinate with other agents or human users. These properties collectively describe a system that decides *how* to retrieve information, rather than one that simply retrieves it.

The ReAct framework, introduced by Yao et al. (2023), provides the theoretical backbone for this class of agent by interleaving verbal reasoning traces with discrete tool actions in a repeating loop. Each cycle consists of three elements: a Thought, which is a free-form reasoning step in which the model assesses its current state and decides on a next action; an Action, which is a structured call to a defined tool with specific arguments; and an Observation, which is the value returned by that tool and appended to the context before the next Thought step. The agent continues cycling until it determines that sufficient information has been gathered to produce a final answer. Yao et al. (2023) demonstrate that this interleaving significantly outperforms either reasoning or acting in isolation, reporting 27.4 exact-match accuracy on HotpotQA and 71% task success on ALFWorld. Critically, the Thought traces are human-readable, making the agent's decision process auditable in a way that the single-pass RAG pipeline is not.

Tool use is the mechanism through which the agent grounds its reasoning in external information sources, and it directly addresses several limitations of plain language model inference. Schick, Dwivedi-Yu, Dessì, Raileanu, Lomeli, Zettlemoyer, Cancedda and Scialom (2023) identify four such limitations: the model's knowledge is bounded by its training cut-off; in-context missteering can cause factually incorrect completions; hallucination degrades reliability on knowledge-intensive tasks; and domain-specific sub-tasks such as arithmetic require capabilities the base model lacks. The filtering criterion that distinguishes a useful tool call from a redundant one is whether the tool's response reduces future generation loss by at least a fixed threshold — a principle that generalises beyond Toolformer's specific implementation to any agent that must decide whether to query an external source (Schick et al., 2023). The ReAct loop overcomes the sequential dependency limitation inherent in single-step tool use by making each Observation available as context for the subsequent Thought, enabling the agent to compose tool calls iteratively across a multi-hop query.

Wang, Ma, Feng, Zhang, Yang, Zhang, Chen, Tang, Chen, Lin, Zhao, Wei and Wen (2023) decompose agent architecture into four interacting modules that map directly onto the system built in this thesis. The profiling module defines the agent's role and constraints, corresponding to the system prompt that instructs the agent to function as an enterprise retrieval assistant with access to source-specific tools. The memory module encompasses both short-term working memory — the in-context accumulation of the running ReAct trace — and long-term semantic memory, implemented here as the per-source document indexes. The planning module governs how the agent sequences its actions; Wang et al. (2023) distinguish single-path planning (Chain-of-Thought), multi-path planning (Tree-of-Thought), and feedback-driven planning (ReAct), with ReAct identified as optimal for multi-hop retrieval tasks where each step must inform the next. The action module defines the typed tool calls available to the agent, corresponding in this thesis to three source-specific retrieval tools: one for GitHub Issues, one for Markdown documentation, and one for PDF whitepapers.

The Cognitive Architectures for Language Agents (CoALA) framework from Sumers, Yao, Narasimhan and Griffiths (2023) provides a formal basis for distinguishing agent-based systems from classical RAG. CoALA partitions agent memory into working memory (the current query, retrieved passages, and running trace), episodic memory (past trajectories), semantic memory (long-term unstructured knowledge), and procedural memory (implicit weights plus explicit prompt templates and tool definitions). Under this taxonomy, classical RAG is a constrained agent that reads from a fixed, read-only semantic memory with no surrounding decision cycle — Sumers et al. (2023) describe retrieval-augmented methods as "retrieving from a semantic memory of unstructured text" and position classical RAG as the degenerate case in which the agent has no planning stage and no adaptive retrieval decision. The thesis agent adds a decision cycle around retrieval, enabling conditional and iterative access to multiple semantic memory stores rather than a single unified index.

Shinn, Cassano, Berman, Gopinath, Narasimhan and Yao (2023) extend the ReAct loop with verbal self-reflection, in which a dedicated self-reflection model generates a textual summary of why a prior attempt failed, storing that summary in episodic memory for the next attempt. The resulting system achieves 97% task success on ALFWorld, compared to 78% for standard ReAct, demonstrating that verbal reinforcement of the decision cycle substantially improves agent performance on complex sequential tasks. This capability is not implemented in the agent evaluated in this thesis, as queries are treated as independent and no learning is persisted between runs; Reflexion

nonetheless represents a natural extension direction that would bring the system closer to a continuously adaptive enterprise retrieval agent.

The contrast between classical RAG and agent-based retrieval is most sharply visible across the evaluation dimensions examined in this thesis. Classical RAG routes all queries through a single unified index, retrieves once per query, and produces a response in a single language model call, resulting in lower latency and lower token expenditure. The agent-based system routes each query to one or more source-specific tools, retrieves conditionally and iteratively based on accumulated observations, and may invoke the language model several times before producing a final response. This iterative structure improves support for multi-hop queries that require synthesising information from multiple documents or sources, but imposes measurable costs in latency and token consumption — a trade-off Gao et al. (2023) identify as central to the design space of advanced retrieval systems. The readable Thought traces produced by the ReAct loop partially offset the higher cost by providing interpretability that the black-box RAG pipeline cannot match.

2.4 Evaluation Methodology

Evaluating retrieval-augmented systems presents a challenge that traditional natural language processing (NLP) benchmarks are not designed to address: the quality of a generated answer depends jointly on retrieval precision and generation faithfulness, and these two failure modes require distinct measurement instruments. The evaluation framework employed in this thesis combines three complementary layers—reference-free retrieval-grounded metrics, reference-dependent semantic similarity measures, and non-quality operational metrics—to produce a comprehensive and fair comparison between the classical RAG pipeline and the agent-based system.

RAGAS: Reference-Free Retrieval-Grounded Evaluation

The RAGAS framework (Es et al., 2023) provides three metrics that assess the pipeline end-to-end without requiring pre-existing ground truth answer annotations. Context Relevance measures whether retrieved passages are genuinely necessary for answering the query. An LLM is prompted to extract only those sentences from the retrieved

context that are essential for a response, yielding a ratio CR $= |S_{\text{extracted}}|/|S_{\text{total}}|$, where a score approaching 1.0 indicates precise, low-noise retrieval. This metric is the most sensitive to retrieval strategy differences between the two systems, though it also shows the weakest alignment with human judgements (0.70 pairwise accuracy), partly due to extraction failures on long passages (Es et al., 2023).

Answer Faithfulness measures the degree to which a generated answer is grounded in the retrieved context, rather than in parametric model knowledge. The metric decomposes the answer into atomic claims, verifies each binary claim against the context, and computes Faithfulness $= |V|/|S|$, where V denotes verified claims and S the total claim count. A score of 1.0 indicates full contextual grounding, while lower scores signal hallucination relative to the retrieved evidence. Validation on the WikiEval benchmark demonstrates 0.95 pairwise accuracy against human judgements, making Faithfulness the most reliable RAGAS metric (Es et al., 2023).

Answer Relevance captures whether the generated response addresses the original query, penalising verbose, incomplete, or off-topic answers. An LLM generates N synthetic paraphrases of the query from the answer text, and the metric computes AR $= \frac{1}{N}\sum_{i=1}^{N} \cos(\mathbf{e}(q), \mathbf{e}(\hat{q}_i))$, where $\mathbf{e}(\cdot)$ denotes the embedding function. Because RAGAS operates without reference answers, it supports evaluation on synthetic corpora where no pre-existing annotations exist (Es et al., 2023). However, RAGAS cannot detect source routing errors: an answer that is fully grounded in a retrieved passage from the wrong source type still achieves a Faithfulness score of 1.0. Source routing accuracy must therefore be measured as a separate, agent-specific metric.

BERTScore: Reference-Dependent Semantic Similarity

Lexical overlap metrics such as BLEU and ROUGE-L were designed for machine translation and summarisation tasks in which the reference text and the candidate response share significant surface form. In a retrieval setting where correct answers may be expressed through diverse valid paraphrases, BLEU produces near-random correlation with human judgements—reaching a Pearson correlation of -0.019 on the COCO image captioning benchmark (Zhang, Kishore, Wu, Weinberger and Artzi, 2020)—and therefore provides insufficient sensitivity for this evaluation.

BERTScore (Zhang et al., 2020) addresses this gap by computing soft token-level matching between candidate and reference using contextual embeddings from RoBERTa-Large. Recall, precision, and F1 are defined as:

$$R_{\text{BERT}} = \frac{1}{|x|} \sum_{i} \max_{j} \mathbf{x}_i^\top \hat{\mathbf{x}}_j \tag{2.1}$$

$$P_{\text{BERT}} = \frac{1}{|\hat{x}|} \sum_{j} \max_{i} \mathbf{x}_i^\top \hat{\mathbf{x}}_j \tag{2.2}$$

$$F_{\text{BERT}} = 2 \cdot \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} \tag{2.3}$$

where $\mathbf{x}_i$ and $\hat{\mathbf{x}}_j$ are the contextual token embeddings of the reference and candidate respectively, optionally weighted by inverse document frequency to down-weight function words. BERTScore achieves a recall correlation of 0.917 under IDF weighting on the same captioning benchmark, and its advantage over BLEU is largest when paraphrase diversity is high (Zhang et al., 2020)—exactly the condition that characterises open-domain RAG evaluation.

BERTScore and RAGAS are complementary rather than redundant. RAGAS Faithfulness is reference-free and measures grounding in retrieved context; BERTScore is reference-dependent and measures semantic proximity to a correct ground truth answer. An answer grounded faithfully in incomplete retrieved content may score highly on Faithfulness yet diverge substantially from the correct answer, a gap that only BERTScore captures. Together, the two instruments cover both retrieval-side and generation-side quality without redundancy.

Four Capability Dimensions from Gao et al.

Beyond aggregate metric scores, Gao et al. (2023) identify four robustness dimensions that characterise the qualitative behaviour of retrieval-augmented systems under realistic operating conditions. Noise robustness concerns whether a system can identify and use the relevant passages within a context that also contains irrelevant or contradictory material. Negative rejection tests whether a system correctly declines to answer when no retrieved passage contains useful information, rather than hallucinating a plausible-sounding response. Information integration measures whether the system can synthesise content from multiple retrieved passages to answer multi-hop queries, a capability that single-pass RAG struggles with by design. Counterfactual robustness

assesses whether a system can identify and discount factual errors present in retrieved passages. These four dimensions directly inform the query taxonomy used in this evaluation: simple factual queries probe baseline retrieval quality, cross-document synthesis queries test information integration, and ambiguous or incomplete queries probe negative rejection and noise robustness.

Synthetic Query Generation and LLM-as-Judge

No pre-existing annotated query set covers the specific proxy corpus—GitHub Issues, Markdown documentation, and PDF whitepapers—used in this thesis. Synthetic QA generation addresses this constraint within a zero-budget academic context: an LLM is prompted to produce candidate question–answer pairs from sampled document passages, and the resulting set is manually reviewed and filtered against quality criteria before use. RAGAS explicitly validates this methodology, demonstrating that reference-free metrics achieve 0.70–0.95 pairwise accuracy with human judgements and that the framework "enables faster evaluation cycles of RAG architectures without relying on ground truth annotations" (Es et al., 2023). The manual review step is essential for academic credibility, as it filters incoherent, unanswerable, or trivially simple items that automated generation produces. Ground truth answers produced during this process additionally serve as the reference for BERTScore computation.

Operational and Agent-Specific Metrics

Quality metrics alone are insufficient for an architectural comparison that must inform practical deployment decisions. Latency is measured as wall-clock time from query submission to complete response, averaged over multiple runs, and includes all LLM API calls and, for the agent, all intermediate tool execution steps. Token cost is derived from input and output token counts in API response objects, converted to an estimated cost per query and per 1,000 queries; the agent incurs structurally higher token consumption due to multi-step reasoning traces. Two agent-specific metrics are tracked in addition: tool call count per query (average and stratified by query type) and source routing accuracy, defined as the proportion of queries for which the agent queries the correct source type relative to a manually labelled ground truth. Implementation complexity is assessed through a structured rubric covering flexibility, debuggability, and extensibility, proxied by objective measures including lines of code per component,

number of configuration parameters, and the time required to onboard an additional data source. Ferrazzi, Cvjeticanin, Piraccini and Giannuzzi (2026) explicitly exclude implementation complexity from their evaluation scope; its inclusion here as a formal dimension constitutes a direct contribution of this thesis.

Table 2.1: Evaluation metrics employed in this thesis, organised by dimension.

Dimension	**Metric**	**Reference-free?**
Context relevance	RAGAS Context Relevance	Yes
Answer faithfulness	RAGAS Faithfulness	Yes
Answer relevance	RAGAS Answer Relevance	Yes
Semantic similarity	BERTScore F_1 (RoBERTa-Large)	No
Latency	Mean wall-clock time per query	—
Token cost	API token counts $\rightarrow$ estimated USD	—
Agent efficiency	Tool calls per query	—
Agent routing	Source selection accuracy	No
Complexity	Rubric score + objective proxies	—

Taken together, these considerations suggest that no single metric is sufficient to characterise the difference between a classical RAG pipeline and an agent-based system, and that a multi-dimensional framework spanning retrieval quality, generation fidelity, operational cost, and structural complexity is necessary to produce findings that are both academically rigorous and practically actionable.

3 System Design

3.1 Shared Data Layer

3.1.1 Data Source Selection and Justification

3.1.2 Unified Document Schema

3.1.3 Ingestion Data Flow

3.2 Classical RAG Pipeline Architecture

3.2.1 Pipeline Stages

3.2.2 Chunking Strategy

3.2.3 Embedding Model and Vector Store

3.2.4 Retrieval and Reranking

3.3 Agent-Based System Architecture

3.3.1 The Agent Loop

3.3.2 Tools and Source Routing

3.3.3 Orchestration Logic

3.4 Comparability Design Decisions

4 Data and Evaluation Setup

4.1 Dataset Construction

4.1.1 Data Collection per Source Type

4.1.2 Preprocessing and Normalisation

4.1.3 Corpus Statistics

4.2 Evaluation Query Set

4.2.1 Synthetic QA Generation

4.2.2 Query Taxonomy

4.2.3 Manual Review and Filtering

4.2.4 Final Query Set Statistics

4.3 Evaluation Metrics Definition

4.3.1 RAGAS Scores

4.3.2 BERTScore and ROUGE-L

4.3.3 Latency and Token Cost

4.3.4 Agent-Specific Metrics

4.3.5 Complexity and Maintainability Rubric

5 Implementation

5.1 Development Environment and Tooling

5.2 RAG Pipeline Implementation

5.2.1 Chunking and Embedding

5.2.2 Retrieval and Reranking

5.2.3 Failure Modes and Resolutions

5.3 Agent Implementation

5.3.1 Agent Loop

5.3.2 Tool Implementations

5.3.3 System Prompt and Routing Behaviour

5.4 Logging and Instrumentation

6 Evaluation Results

6.1 Retrieval and Generation Quality

6.2 Latency and Cost

6.3 Implementation Complexity and Maintainability

6.4 Robustness Observations

7 Discussion

7.1 Answering the Research Questions

7.2 Trade-Off Analysis

7.3 Limitations

7.4 Connection to Prior Work

8 Conclusion

8.1 Summary of Findings

8.2 Practical Recommendation

8.3 Future Work

Figures

Tables

2.1 Evaluation metrics employed in this thesis, organised by dimension. . . 17

Listings

References

Asai, A., Wu, Z., Wang, Y., Sil, A. and Hajishirzi, H. (2023), 'SELF-RAG: Learning to retrieve, generate, and critique through self-reflection', *arXiv preprint arXiv:2310.11511* .
URL: *https://arxiv.org/abs/2310.11511*

Es, S., James, J., Anke, L. E. and Schockaert, S. (2023), 'RAGAS: Automated evaluation of retrieval augmented generation', *arXiv preprint arXiv:2309.15217* .
URL: *https://arxiv.org/abs/2309.15217*

Ferrazzi, P., Cvjeticanin, M., Piraccini, A. and Giannuzzi, D. (2026), 'Is agentic RAG worth it? an experimental comparison of RAG approaches', *arXiv preprint arXiv:2601.07711* .
URL: *https://arxiv.org/abs/2601.07711*

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J. and Wang, H. (2023), 'Retrieval-augmented generation for large language models: A survey', *arXiv preprint arXiv:2312.10997* .
URL: *https://arxiv.org/abs/2312.10997*

Jiang, Z., Xu, F. F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J. and Neubig, G. (2023), 'Active retrieval augmented generation', *arXiv preprint arXiv:2305.06983* .
URL: *https://arxiv.org/abs/2305.06983*

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S. and Kiela, D. (2020), Retrieval-augmented generation for knowledge-intensive NLP tasks, *in* 'Advances in Neural Information Processing Systems', Vol. 33, pp. 9459–9474.
URL: *https://arxiv.org/abs/2005.11401*

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N. and Scialom, T. (2023), Toolformer: Language models can teach themselves to use tools, *in* 'Advances in Neural Information Processing Systems'.
URL: *https://arxiv.org/abs/2302.04761*

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K. and Yao, S. (2023), Reflexion: Language agents with verbal reinforcement learning, *in* 'Advances in Neural Information Processing Systems'.
URL: *https://arxiv.org/abs/2303.11366*

Sumers, T. R., Yao, S., Narasimhan, K. and Griffiths, T. L. (2023), 'Cognitive architectures for language agents', *Transactions on Machine Learning Research* .
URL: *https://arxiv.org/abs/2309.02427*

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z. and Wen, J.-R. (2023), 'A survey on large language model based autonomous agents', *arXiv preprint arXiv:2308.11432* .
URL: *https://arxiv.org/abs/2308.11432*

Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., Zheng, R., Fan, X., Wang, X., Xiong, L., Liu, Q., Zhou, Y., Wang, W., Jiang, C., Zou, Y., Liu, X., Yin, Z., Dou, S., Weng, R., Cheng, W., Zhang, Q., Qin, W., Zheng, Y., Qiu, X., Huang, X. and Gui, T. (2023), 'The rise and potential of large language model based agents: A survey', *arXiv preprint arXiv:2309.07864* .
URL: *https://arxiv.org/abs/2309.07864*

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. and Cao, Y. (2023), ReAct: Synergizing reasoning and acting in language models, *in* 'International Conference on Learning Representations'.
URL: *https://arxiv.org/abs/2210.03629*

Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q. and Artzi, Y. (2020), BERTScore: Evaluating text generation with BERT, *in* 'International Conference on Learning Representations'.
URL: *https://arxiv.org/abs/1904.09675*